

```

import { useState, useEffect, useRef } from "react";

const CATEGORIES = [
  { id: "adventure", label: "Adventure", emoji: "🏔️", color: "#E8825A" },
  { id: "travel", label: "Travel", emoji: "✈️", color: "#5A9BE8" },
  { id: "creative", label: "Creative", emoji: "🎨", color: "#C45AE8" },
  { id: "food", label: "Food", emoji: "🍽️", color: "#E8C45A" },
  { id: "learn", label: "Learn", emoji: "📖", color: "#5AE8A3" },
  { id: "connect", label: "Connect", emoji: "❤️", color: "#E85A7A" },
];

const WORLD_LIST = [
  { title: "See the Northern Lights in Iceland", category: "travel",
  { title: "Hike to Machu Picchu, Peru", category: "adventure",
  { title: "Watch sunrise over Angkor Wat, Cambodia", category: "travel",
  { title: "Safari in the Serengeti, Tanzania", category: "adventure",
  { title: "Swim in the Blue Grotto, Capri", category: "travel",
  { title: "Walk the Camino de Santiago, Spain", category: "adventure",
  { title: "See cherry blossoms in Kyoto, Japan", category: "travel",
  { title: "Dive the Great Barrier Reef, Australia", category: "adventure",
  { title: "Watch the wildebeest migration, Kenya", category: "travel",
  { title: "Climb Mount Kilimanjaro, Tanzania", category: "adventure",
  { title: "Float in the Dead Sea, Jordan", category: "travel",
  { title: "See the Taj Mahal at sunrise, India", category: "travel",
  { title: "Ride the Trans-Siberian Railway", category: "travel",
  { title: "Explore the Amazon Rainforest, Brazil", category: "adventure",
  { title: "Witness a total solar eclipse", category: "travel",
  { title: "Skydive over the Swiss Alps", category: "adventure",
  { title: "Sail through the Greek Islands", category: "travel",
  { title: "Trek to Everest Base Camp, Nepal", category: "adventure",
  { title: "Walk the Great Wall of China", category: "travel",
  { title: "Attend Carnival in Rio de Janeiro", category: "connect",
  { title: "Bungee jump in Queenstown, New Zealand", category: "adventure",
  { title: "See the Pyramids of Giza, Egypt", category: "travel",
  { title: "Road trip Route 66, USA", category: "adventure",
  { title: "Experience Oktoberfest in Munich", category: "food",
  { title: "See the midnight sun in Norway", category: "travel",
  { title: "Explore Patagonia, Argentina & Chile", category: "adventure",
  { title: "Eat at a 3-Michelin-star restaurant", category: "food",
  { title: "Learn to surf in Bali, Indonesia", category: "learn",
  { title: "Take a cooking class in Tuscany, Italy", category: "food",
  { title: "Go whale watching in the Azores", category: "travel",
  { title: "Stargaze in the Atacama Desert, Chile", category: "travel",
  { title: "Kayak through Halong Bay, Vietnam", category: "adventure",

```

```

    { title: "Visit the Louvre & see the Mona Lisa",          category: "creative",
    { title: "See a Broadway show in New York City",          category: "creative",
    { title: "Run with the bulls in Pamplona, Spain",          category: "adventure",
    { title: "Stay in an overwater bungalow, Maldives",        category: "travel",
    { title: "Ride a hot air balloon over Cappadocia",          category: "travel",
    { title: "Volunteer abroad for a month",                    category: "connect",
    { title: "Learn a new language to fluency",                 category: "learn",
    { title: "Write and publish a book",                         category: "creative",
    { title: "See a rocket launch in person",                   category: "learn",
    { title: "Backpack solo through Southeast Asia",            category: "adventure",
    { title: "Eat street food through Tokyo for a week",        category: "food",
    { title: "Take a masterclass with a world-class chef",      category: "food",
    { title: "Perform on stage – music, comedy or theatre",     category: "creative",
    { title: "Plant 100 trees",                                  category: "connect",
    { title: "Learn to ride a motorcycle",                       category: "learn",
    { title: "Attend a Grand Slam tennis final",                 category: "connect",
    { title: "Attend the Olympic Games",                         category: "connect",
    { title: "Visit all 7 continents",                           category: "travel",
  ];

```

```

const UK_LIST = [
  { title: "Hike to the top of Ben Nevis, Scotland",          category: "adventur
  { title: "Watch the Changing of the Guard, London",          category: "travel",
  { title: "Walk the Jurassic Coast, Dorset",                  category: "adventur
  { title: "See Stonehenge at sunrise, Wiltshire",              category: "travel",
  { title: "Explore the Giant's Causeway, N. Ireland",          category: "travel",
  { title: "Attend the Edinburgh Fringe Festival",              category: "creative
  { title: "Swim in the Fairy Pools, Isle of Skye",            category: "adventur
  { title: "Watch a match at Wembley Stadium",                  category: "connect"
  { title: "See the White Cliffs of Dover",                     category: "travel",
  { title: "Cruise the Norfolk Broads by boat",                 category: "adventur
  { title: "Walk Hadrian's Wall end to end",                     category: "adventur
  { title: "Attend the Chelsea Flower Show, London",            category: "creative
  { title: "Explore the Cairngorms in winter",                  category: "adventur
  { title: "Have afternoon tea at The Ritz, London",            category: "food",
  { title: "See puffins on Skomer Island, Wales",               category: "travel",
  { title: "Watch the Proms at the Royal Albert Hall",          category: "creative
  { title: "Cycle the South Downs Way",                          category: "adventur
  { title: "Ride the Snowdon Mountain Railway, Wales",          category: "travel",
  { title: "Visit every county in England",                     category: "travel",
  { title: "See the Milky Way from the Brecon Beacons",         category: "travel",
];

```

```

const STORAGE_KEY = "bucket-list-items-v2";

```

```

function getCategory(id) {
  return CATEGORIES.find((c) => c.id === id) || CATEGORIES[0];
}

```

```
}
```

```
export default function BucketList() {
  const [items, setItems] = useState([]);
  const [view, setView] = useState("list");
  const [selectedItem, setSelectedItem] = useState(null);
  const [filter, setFilter] = useState("all");
  const [newItem, setNewItem] = useState({ title: "", category: "ad
  const [loaded, setLoaded] = useState(false);
  const [justCompleted, setJustCompleted] = useState(null);
  const [inspireTab, setInspireTab] = useState("world");
  const [inspireCat, setInspireCat] = useState("all");
  const [addedTitles, setAddedTitles] = useState(new Set());
  const titleRef = useRef(null);

  useEffect(() => {
    (async () => {
      try {
        const result = await window.storage.get(STORAGE_KEY);
        if (result?.value) {
          const saved = JSON.parse(result.value);
          setItems(saved);
          setAddedTitles(new Set(saved.map((i) => i.title)));
        }
      } catch {}
      setLoaded(true);
    })();
  }, []);

  useEffect(() => {
    if (!loaded) return;
    window.storage.set(STORAGE_KEY, JSON.stringify(items)).catch(() => {});
  }, [items, loaded]);

  useEffect(() => {
    if (view === "add" && titleRef.current) setTimeout(() => titleRef.current?.fo
  }, [view]);

  const commitAdd = (overrides = {}) => {
    const base = { ...newItem, ...overrides };
    if (!base.title?.trim()) return;
    const item = {
      id: Date.now().toString() + Math.random().toString(36).slice(2),
      ...base,
      title: base.title.trim(),
      completed: false,
      createdAt: new Date().toISOString(),
    };
  };
}
```

```

    };
    setItems((prev) => [item, ...prev]);
    setAddedTitles((prev) => new Set([...prev, item.title]));
    if (!overrides.title) {
      setNewItem({ title: "", category: "adventure", note: "", priority: "medium"
        setView("list");
      }
    };

const addSuggestion = (s) => {
  if (addedTitles.has(s.title)) return;
  commitAdd({ title: s.title, category: s.category, priority: s.priority, note:
};

const toggleComplete = (id) => {
  setItems((prev) =>
    prev.map((item) => {
      if (item.id !== id) return item;
      const completed = !item.completed;
      if (completed) setJustCompleted(id);
      return { ...item, completed, completedAt: completed ? new Date().toISOStr
    })
  );
  setTimeout(() => setJustCompleted(null), 1200);
};

const deleteItem = (id) => {
  const dying = items.find((i) => i.id === id);
  if (dying) setAddedTitles((prev) => { const n = new Set(prev); n.delete(dying
  setItems((prev) => prev.filter((i) => i.id !== id));
  setView("list");
};

const filtered = items.filter((item) =>
  filter === "done" ? item.completed : filter === "todo" ? !item.completed : tr
);
const doneCount = items.filter((i) => i.completed).length;
const progress = items.length ? Math.round((doneCount / items.length) * 100)
const activeList = inspireTab === "world" ? WORLD_LIST : UK_LIST;
const inspireItems = inspireCat === "all" ? activeList : activeList.filter((w)
const addedWorld = WORLD_LIST.filter((w) => addedTitles.has(w.title)).length;
const addedUK = UK_LIST.filter((w) => addedTitles.has(w.title)).length;

if (!loaded) return <div style={S.loading}><div style={S.dot} /></div>;

return (
  <div style={S.phone}>

```

```

<div style={S.app}>

  {/* HEADER */}
  <div style={S.header}>
    <div style={S.headerTop}>
      <div>
        <div style={S.headerLabel}>MY</div>
        <div style={S.headerTitle}>Bucket List</div>
      </div>
      <div style={{ display:"flex", gap:10 }}>
        <button onClick={() => { setView("inspire"); setInspireTab("world")
        <button onClick={() => setView("add")} style={S.addBtn}><span style
        </div>
      </div>

    {items.length > 0 && (
      <div style={S.progressSection}>
        <div style={S.progressRow}>
          <span style={S.progressLabel}><doneCount> of <items.length> compl
          <span style={S.progressPct}><progress>%</span>
        </div>
        <div style={S.progressTrack}>
          <div style={{ ...S.progressFill, width:`${progress}%`, background
          </div>
        </div>
      </div>
    )}

    <div style={S.filters}>
      {[ "all", "todo", "done" ].map((f) => (
        <button key={f} onClick={() => setFilter(f)}
          style={{ ...S.filterBtn, ...(filter===f ? S.filterActive : {}) }}
          {f==="all" ? "All" : f==="todo" ? "To Do" : "Done"}
        </button>
      ))}
    </div>
  </div>

  {/* LIST */}
  <div style={S.content}>
    {filtered.length === 0 && (
      <div style={S.empty}>
        <div style={S.emptyEmoji}><filter=== "done" ? "🎉" : "🌟"></div>
        <div style={S.emptyText}>
          {filter=== "done" ? "Nothing completed yet" : filter=== "todo" ? "N
        </div>
      <div style={{ display:"flex", flexDirection:"column", gap:10, ali

```

```

        <button onClick={() => setView("inspire")} style={S.inspireEmpt
        <button onClick={() => setView("add")} style={S.emptyBtn}>✎ Ac
    </div>
  )}
</div>
)}
{filtered.map((item) => {
  const cat    = getCategory(item.category);
  const flash = justCompleted === item.id;
  return (
    <div key={item.id} style={{ ...S.card, opacity:item.completed?0.68:
      onClick={() => { setSelectedItem(item); setView("detail"); }}>
      <button style={{ ...S.checkCircle, borderColor:item.completed?cat
        onClick={(e) => { e.stopPropagation(); toggleComplete(item.id);
          {item.completed && <span style={S.checkMark}>✓</span>}}
      </button>
      <div style={S.cardBody}>
        <div style={S.cardTitle}>{item.title}</div>
        <div style={S.cardMeta}>
          <span style={{ ...S.catBadge, background:cat.color+"22", colo
            {item.priority==="high" && <span style={S.highBadge}>🔥 High<
          </div>
        </div>
        <div style={S.cardArrow}>></div>
      </div>
    );
  )}}
</div>

{/* INSPIRE SHEET */}
{view==="inspire" && (
  <div style={S.sheet} onClick={() => setView("list")}>
    <div style={S.sheetContent} onClick={(e) => e.stopPropagation()}>
      <div style={S.sheetHandle} />

      {/* Tabs */}
      <div style={S.tabRow}>
        <button onClick={() => { setInspireTab("world"); setInspireCat("a
          style={{ ...S.tab, ...(inspireTab==="world" ? S.tabActive : {})}
          🌐 World <span style={S.tabCount}>{addedWorld}/50</span>
        </button>
        <button onClick={() => { setInspireTab("uk"); setInspireCat("all"
          style={{ ...S.tab, ...(inspireTab==="uk" ? S.tabActive : {})} }}
          🇬🇧 UK <span style={S.tabCount}>{addedUK}/20</span>
        </button>
      </div>
    </div>
  )}
)

```

```

    {/* Category chips */}
    <div style={S.chipRow}>
      <button onClick={() => setInspireCat("all")}
        style={{ ...S.chip, ...(inspireCat==="all" ? S.chipActive : {})}
        {CATEGORIES.map((cat) => (
          <button key={cat.id} onClick={() => setInspireCat(cat.id)}
            style={{ ...S.chip, ...(inspireCat===cat.id ? { ...S.chipActi
              {cat.emoji}
            }
          )}}
        )}}
    </div>

    {/* Items */}
    <div style={S.inspireList}>
      {inspireItems.map((w, i) => {
        const cat = getCategory(w.category);
        const added = addedTitles.has(w.title);
        return (
          <div key={i} style={{ ...S.inspireCard, opacity:added?0.5:1,
            <div style={{ ...S.inspireIcon, background:cat.color+"1A" }
              <div style={S.inspireBody}>
                <div style={S.inspireTitle}>{w.title}</div>
                <div style={{ display:"flex", gap:5, marginTop:3 }}>
                  <span style={{ ...S.catBadge, background:cat.color+"18"
                    {w.priority==="high" && <span style={{ fontSize:10, col
                  </div>
                </div>
              <button onClick={() => addSuggestion(w)} disabled={added}
                style={{ ...S.inspireAddBtn, background:added?"rgba(90,23
                  {added ? "✓" : "+"}
              </button>
            </div>
          );
        )}}
      </div>
    </div>
  </div>
)}

{/* ADD SHEET */}
{view==="add" && (
  <div style={S.sheet} onClick={() => setView("list")}>
    <div style={S.sheetContent} onClick={(e) => e.stopPropagation()}>
      <div style={S.sheetHandle} />
      <div style={S.sheetTitle}>New Dream</div>
      <input ref={titleRef} style={S.input} placeholder="What do you want
        value={newItem.title}

```

```

        onChange={ (e) => setNewItem({ ...newItem, title:e.target.value })
        onKeyDown={ (e) => e.key==="Enter" && commitAdd() }
        maxLength={100} />
<div style={S.fieldLabel}>Category</div>
<div style={S.catGrid}>
    {CATEGORIES.map((cat) => (
        <button key={cat.id} onClick={() => setNewItem({ ...newItem, ca
            style={{ ...S.catOption, borderColor:newItem.category===cat.i
                <span style={{ fontSize:20 }}>{cat.emoji}</span>
                <span style={{ ...S.catOptionLabel, color:newItem.category===
            </button>
        )}}
    )}}
</div>
<div style={S.fieldLabel}>Priority</div>
<div style={S.priorityRow}>
    {[ "low", "medium", "high" ].map((p) => (
        <button key={p} onClick={() => setNewItem({ ...newItem, priorit
            style={{ ...S.priorityBtn, background:newItem.priority===p ?
                {p==="high"? "🔥":p==="medium"? "⚡": "zZ"} } {p}
            </button>
        )}}
    )}}
</div>
<div style={S.fieldLabel}>Note (optional)</div>
<textarea style={S.textarea} rows={3} placeholder="Why does this ma
    value={newItem.note}
    onChange={ (e) => setNewItem({ ...newItem, note:e.target.value }) }
    maxLength={300} />
<button onClick={() => commitAdd()} style={{ ...S.saveBtn, opacity:
    Add to List
</button>
</div>
</div>
)}

{/* DETAIL SHEET */}
{view==="detail" && selectedItem && (() => {
    const item = items.find((i) => i.id===selectedItem.id) || selectedItem;
    const cat = getCategory(item.category);
    return (
        <div style={S.sheet} onClick={() => setView("list")}>
            <div style={S.sheetContent} onClick={(e) => e.stopPropagation()}>
                <div style={S.sheetHandle} />
                <div style={{ marginBottom:14 }}>
                    <span style={{ ...S.catBadge, background:cat.color+"22", color:
                        {cat.emoji} {cat.label}
                    </span>
                </div>
            </div>
        </div>
    )
    )}

```



```

<div style={S.detailTitle}>{item.title}</div>
{item.note && <div style={S.detailNote}>{item.note}</div>}
<div style={S.detailMeta}>
  <div style={S.metaRow}>
    <span style={S.metaKey}>Priority</span>
    <span style={S.metaVal}>{item.priority=== "high"? "🔥 High":ite
  </div>
  <div style={S.metaRow}>
    <span style={S.metaKey}>Added</span>
    <span style={S.metaVal}>{new Date(item.createdAt).toLocaleDat
  </div>
  {item.completed && item.completedAt && (
    <div style={S.metaRow}>
      <span style={S.metaKey}>Completed</span>
      <span style={{ ...S.metaVal, color:"#5AE8A3" }}>{new Date(i
    </div>
  )}
</div>
<button onClick={() => { toggleComplete(item.id); setView("list")
  style={{ ...S.saveBtn, marginBottom:10, background:item.complet
    {item.completed ? "↺ Mark Incomplete" : "✓ Mark Complete!"}
  </button>
  <button onClick={() => deleteItem(item.id)} style={S.deleteBtn}>D
  </div>
</div>

  );
  }) ()}
</div>

<style>{`
  @import url('https://fonts.googleapis.com/css2?family=Playfair+Display:wg
  *(box-sizing:border-box;margin:0;padding:0;-webkit-tap-highlight-color:tr
  button{cursor:pointer;border:none;font-family:'DM Sans',sans-serif;}
  input,textarea{font-family:'DM Sans',sans-serif;}
  textarea{resize:none;}
  ::-webkit-scrollbar{display:none;}
  @keyframes slideUp{from{transform:translateY(100%)}to{transform:translate
  @keyframes fadeIn{from{opacity:0}to{opacity:1}}
  @keyframes pulse{0%,100%{transform:scale(1)}50%{transform:scale(1.15)}}
  `}</style>
</div>

  );
}

const S = {
  phone:{ display:"flex", justifyContent:"center", minHeight:"100vh", background:
  app:{ width:"100%", maxWidth:430, minHeight:"100vh", background:"linear-gradien

```

```

loading:{ display:"flex", justifyContent:"center", alignItems:"center", height:
dot:{ width:12, height:12, background:"#E8825A", borderRadius:"50%", animation:

header:{ padding:"52px 20px 16px", background:"linear-gradient(180deg,#1A1A22 0
headerTop:{ display:"flex", justifyContent:"space-between", alignItems:"flex-st
headerLabel:{ fontSize:11, letterSpacing:"0.2em", color:"rgba(255,255,255,0.3)"
headerTitle:{ fontFamily:"'Playfair Display',serif", fontSize:34, fontWeight:90
globeBtn:{ width:44, height:44, borderRadius:"50%", background:"rgba(255,255,25
addBtn:{ width:44, height:44, borderRadius:"50%", background:"linear-gradient(1

progressSection:{ marginBottom:16 },
progressRow:{ display:"flex", justifyContent:"space-between", marginBottom:6 },
progressLabel:{ fontSize:12, color:"rgba(255,255,255,0.38)" },
progressPct:{ fontSize:12, color:"rgba(255,255,255,0.58)", fontWeight:500 },
progressTrack:{ height:4, background:"rgba(255,255,255,0.08)", borderRadius:2,
progressFill:{ height:"100%", borderRadius:2, transition:"width 0.5s cubic-bezi

filters:{ display:"flex", gap:8 },
filterBtn:{ padding:"6px 16px", borderRadius:20, background:"rgba(255,255,255,0
filterActive:{ background:"rgba(232,130,90,0.15)", color:"#E8825A" },

content:{ padding:"12px 16px 120px", overflowY:"auto", maxHeight:"calc(100vh -

empty:{ display:"flex", flexDirection:"column", alignItems:"center", justifyCon
emptyEmoji:{ fontSize:52, animation:"pulse 2s infinite" },
emptyText:{ color:"rgba(255,255,255,0.3)", fontSize:16, textAlign:"center" },
emptyBtn:{ background:"none", color:"#E8825A", fontSize:14, fontWeight:500, bor
inspireEmptyBtn:{ background:"rgba(255,255,255,0.07)", color:"rgba(255,255,255,

card:{ display:"flex", alignItems:"center", gap:14, padding:14, background:"rgb
checkCircle:{ width:28, height:28, borderRadius:"50%", border:"2px solid", flex
checkMark:{ fontSize:13, color:"#111", fontWeight:700, lineHeight:1 },
cardBody:{ flex:1, minWidth:0 },
cardTitle:{ fontSize:15, fontWeight:500, color:"rgba(255,255,255,0.88)", margin
cardMeta:{ display:"flex", gap:6, flexWrap:"wrap" },
catBadge:{ fontSize:11, padding:"3px 8px", borderRadius:8, fontWeight:500 },
highBadge:{ fontSize:11, padding:"3px 8px", borderRadius:8, background:"rgba(23
cardArrow:{ fontSize:20, color:"rgba(255,255,255,0.15)", lineHeight:1 },

sheet:{ position:"fixed", inset:0, background:"rgba(0,0,0,0.72)", backdropFilde
sheetContent:{ width:"100%", maxWidth:430, margin:"0 auto", background:"linear-
sheetHandle:{ width:40, height:4, background:"rgba(255,255,255,0.15)", borderRa
sheetTitle:{ fontFamily:"'Playfair Display',serif", fontSize:26, fontWeight:700

tabRow:{ display:"flex", gap:8, marginBottom:16 },
tab:{ flex:1, padding:"10px 8px", borderRadius:12, background:"rgba(255,255,255
tabActive:{ background:"rgba(232,130,90,0.15)", color:"#E8825A", borderColor:"r

```

```

tabCount:{ fontSize:11, opacity:0.7, fontWeight:400 },

chipRow:{ display:"flex", gap:6, overflowX:"auto", marginBottom:16, paddingBott
chip:{ padding:"5px 12px", borderRadius:16, background:"rgba(255,255,255,0.06)"
chipActive:{ background:"rgba(232,130,90,0.12)", color:"#E8825A", borderColor:"

inspireList:{ display:"flex", flexDirection:"column", gap:8 },
inspireCard:{ display:"flex", alignItems:"center", gap:12, padding:12, backgrou
inspireIcon:{ width:36, height:36, borderRadius:10, display:"flex", alignItems:
inspireBody:{ flex:1, minWidth:0 },
inspireTitle:{ fontSize:13, fontWeight:500, color:"rgba(255,255,255,0.85)", lin
inspireAddBtn:{ width:32, height:32, borderRadius:"50%", border:"1.5px solid",

input:{ width:"100%", background:"rgba(255,255,255,0.06)", border:"1px solid rg
fieldLabel:{ fontSize:11, fontWeight:600, letterSpacing:"0.12em", color:"rgba(2
catGrid:{ display:"grid", gridTemplateColumns:"1fr 1fr 1fr", gap:8, marginBotto
catOption:{ display:"flex", flexDirection:"column", alignItems:"center", gap:4,
catOptionLabel:{ fontSize:11, fontWeight:500, transition:"color 0.2s" },
priorityRow:{ display:"flex", gap:8, marginBottom:20 },
priorityBtn:{ flex:1, padding:"10px 8px", borderRadius:10, fontSize:12, fontWei
textarea:{ width:"100%", background:"rgba(255,255,255,0.06)", border:"1px solid
saveBtn:{ width:"100%", padding:16, borderRadius:14, background:"linear-gradien

detailTitle:{ fontFamily:"'Playfair Display',serif", fontSize:28, fontWeight:70
detailNote:{ fontSize:15, color:"rgba(255,255,255,0.48)", lineHeight:1.6, margi
detailMeta:{ marginBottom:24, display:"flex", flexDirection:"column", gap:10 },
metaRow:{ display:"flex", justifyContent:"space-between", alignItems:"center",
metaKey:{ fontSize:13, color:"rgba(255,255,255,0.32)" },
metaVal:{ fontSize:13, color:"rgba(255,255,255,0.7)", fontWeight:500 },
deleteBtn:{ width:"100%", padding:14, borderRadius:14, background:"rgba(232,90,
};

```